

Resolución del Práctico 2

Compilación, linking y layout de memoria de un proceso

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento es la resolución comentada del Práctico 2 de Sistemas Operativos (UNRC, 2026), centrado en el *toolchain* de C: las etapas de pre-procesado, compilación, ensamblado y linking, junto con la diferencia entre **bibliotecas estáticas y dinámicas** y el **layout en memoria** de un proceso (text, data, bss, heap, stack). El objetivo es que un estudiante pueda leer este PDF de corrido y entender qué hace cada herramienta y por qué hace falta. Los comandos fueron ejecutados sobre macOS arm64 (formato Mach-O); el razonamiento es portable: en GNU/Linux las mismas ideas se manifiestan sobre ELF, con `readelf/ldd/objdump` donde acá usamos `otool/nm/objdump`.

Índice

1. Introducción: el toolchain y la memoria de un proceso	1
2. Ejercicio 1 — Pre-procesado, assembly y objeto de <code>hello.c</code>	4
3. Ejercicio 2 — Compilar <code>main.c</code> a <code>main.o</code>	5
4. Ejercicio 3 — Análisis de <code>main.o</code> y <code>hello.o</code>	6
5. Ejercicio 4 — Generar el ejecutable <code>myprog</code>	10
6. Ejercicio 5 — Análisis assembly de <code>myprog</code>	10
7. Ejercicio 6 — Variables locales: el stack	12
8. Ejercicio 7 — Biblioteca estática <code>libhello.a</code>	14
9. Ejercicio 8 — Biblioteca dinámica y <code>dlopen</code>	16
10. Ejercicio 9 — Automatización con <code>Makefile</code>	18
Cierre	21
Referencias	21

1. Introducción: el toolchain y la memoria de un proceso

Programar en C parece resumirse a un único comando: `gcc -o myprog main.c hello.c`. Detrás de esa línea se ejecuta una secuencia ordenada de programas que se conoce como *toolchain*, y el resultado final —el ejecutable— se organiza en memoria con un layout muy concreto. Antes de los ejercicios conviene fijar el vocabulario, porque las preguntas del práctico se entienden mucho mejor si se ve el panorama completo.

Las cuatro etapas del toolchain

Las Notas del curso descomponen la compilación en cuatro etapas:

1. **Pre-procesado (cpp)**. Expande las directivas `#include`, `#define` y demás macros. Salida: `.i` (C ya expandido, todavía texto). Se invoca con `gcc -E`.
2. **Compilación (cc1)**. Traduce el C pre-procesado a *assembly* para la arquitectura objetivo. Salida: `.s`. Se invoca con `gcc -S`.
3. **Ensamblado (as)**. Convierte el assembly en un *archivo objeto* binario (`.o`) en el formato del SO: **ELF** en GNU/Linux, **Mach-O** en macOS, **COFF/PE** en Windows. Se invoca con `gcc -c`.
4. **Linking (ld)**. Combina archivos objeto y bibliotecas, recalcula direcciones y resuelve referencias externas para producir el ejecutable final.

Las opciones `-E`, `-S` y `-c` detienen el proceso después de la primera, segunda y tercera etapa respectivamente.

¿Qué hay dentro de un archivo objeto?

Un `.o` no es código binario plano: es un contenedor con varias **secciones**:

- **.text**: código máquina ejecutable.
- **.data**: variables globales/estáticas con valor inicial distinto de cero.
- **.bss**: variables globales/estáticas inicializadas a cero (no ocupan lugar en disco, solo se reservan los bytes en memoria).
- **.rodata**: constantes y literales de solo lectura. En Mach-O, los strings caen en la sección `__TEXT, __cstring`.
- **Tabla de símbolos**: lista las funciones y variables definidas o referenciadas por el módulo. Cada símbolo es *local*, *global definido* o **UND** (no resuelto).
- **Tabla de reubicación**: indica qué instrucciones tienen operandos pendientes de completar y a qué símbolo se refieren. Es el mapeo `reloc(addr, tipo) → símbolo` que mencionan las notas.

¿Qué hace exactamente el linker?

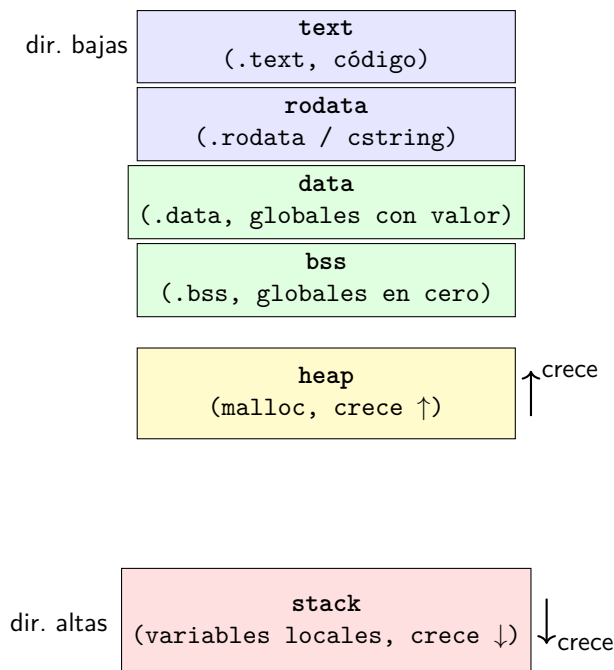
El linker realiza dos tareas conceptuales:

1. Concatenar las secciones de los objetos (`.text+.text+...`) y *recalcular* las direcciones absolutas.
2. Resolver las referencias externas, completando las instrucciones que la tabla de reubicación marcaba como pendientes.

Las llamadas a funciones definidas en el mismo ejecutable se resuelven en **linking estático**; las que viven en una biblioteca compartida (como `printf` en la `libc`) se resuelven en **linking dinámico**, en tiempo de *carga* o de *primera llamada*, mediante *stubs* y la *Global Offset Table* (GOT).

Layout en memoria de un proceso

Una vez que el ejecutable se carga en memoria, el espacio de direcciones del proceso queda organizado en regiones bien definidas. Las secciones del archivo objeto se mapean (con permisos de lectura, escritura, ejecución) a estas regiones del proceso:



Cómo mapea cada sección al proceso

- `.text` y `.rodata` se mapean como **r-x** y **r--** (no escribibles).
- `.data` y `.bss` como **rw-**.
- El **stack** crece hacia direcciones *bajas*: cada `call` empuja un nuevo *stack frame* hacia abajo.
- El **heap** crece hacia direcciones *altas*: cada `malloc()` pide al kernel más espacio vía `brk` o `mmap`.
- Stack y heap crecen uno hacia el otro pero nunca se tocan: el SO los aísla con páginas de guarda.

Bibliotecas estáticas vs dinámicas

Es la dicotomía central del práctico:

- Una biblioteca **estática** (`.a`) es un archivo `ar` con varios `.o` y un índice de símbolos. El linker copia el código necesario *dentro* del ejecutable. Resultado: un binario más grande pero autocontenido.
- Una biblioteca **dinámica** (`.so/.dylib/.dll`) vive en un archivo aparte. El ejecutable solo guarda una referencia y el *loader* del SO la mapea en el proceso al arrancar (o bajo demanda con `dlopen`). Si varios procesos la usan, *una sola copia* en memoria los sirve a todos.

Una tabla resumida (los detalles aparecen en los ejercicios 7 y 8):

	Estática (<code>.a</code>)	Dinámica (<code>.so/.dylib</code>)
Cuándo se resuelven los símbolos	En tiempo de <i>linking</i>	En tiempo de <i>carga</i> o primera llamada
Qué queda en el ejecutable	Una copia del código de la biblioteca	Solo una referencia (<code>stub + GOT</code>)
Tamaño del binario	Más grande	Más chico
Actualizar la biblioteca	Hay que relinkear	Basta con reemplazar el archivo
Memoria entre procesos	Una copia por proceso	Una sola copia compartida
Sintaxis típica	<code>ar rcs libfoo.a a.o b.o</code>	<code>gcc -fPIC -shared</code>

Con esto en la cabeza, los ejercicios del práctico se vuelven una visita guiada por cada uno de estos componentes.

2. Ejercicio 1 — Pre-procesado, assembly y objeto de `hello.c`

Enunciado. Tomar el archivo `hello.c`, generar las salidas intermedias del compilador (pre-procesado, assembly y archivo objeto) y entender qué información contiene cada una.

Programa fuente:

```
1  /* hello.c */
2  #define hi "Hello world"
3
4  char *hello(void)
5  {
6      return hi;
7  }
```

Análisis. `hi` no es una variable: es una **macro**. El preprocesador la reemplaza textualmente por la cadena `"Hello world"` antes de que el compilador siquiera vea el código. Por eso `hi` no va a aparecer en ningún archivo posterior al `.i`.

1.a Pre-procesado (`hello.i`)

Procedimiento:

```
gcc -E hello.c -o hello.i
```

Resultado. El contenido relevante de `hello.i` es:

```
1  # 1 "hello.c"
2  # 1 "<built-in>" 1
3  ...
4  char *hello(void)
5  {
6      return "Hello world";
7  }
```

Conclusión. El identificador `hi` desapareció: la macro fue expandida y reemplazada por el literal `"Hello world"`. Las líneas `# n "archivo"` son marcadores que el preprocesador deja para que los mensajes de error apunten al archivo y línea originales.

1.b Assembly (`hello.s`)

Procedimiento:

```
gcc -S hello.c -o hello.s
```

Resultado:

```
    .section __TEXT,__text,regular,pure_instructions
    .globl _hello
    .p2align 2
_hello:
    adrp x0, 1_._str@PAGE
```

```

        add x0, x0, l_.str@PAGEOFF
        ret
        .section __TEXT,__cstring,cstring_literals
l_.str:
        .asciz "Hello world"

```

Cómo leerlo, línea por línea:

- `__TEXT,__text` contiene el código de `hello`. La etiqueta `_hello` (con underscore delante, convención macOS) es global: la verá el linker.
- `__TEXT,__cstring` contiene el literal, identificado por la etiqueta local `l_.str`. En ELF esto sería la sección `.rodata`.
- ARM64 codifica direcciones en páginas de 4 KiB, por eso la dirección del literal se arma en dos pasos: `adrp` carga la dirección base de la página, y `add` suma el offset dentro de ella.
- El valor de retorno se deposita en `x0` (convención AAPCS64) y la función vuelve con `ret`, que salta a la dirección guardada en el *Link Register* `x30`.

1.c Archivo objeto (`hello.o`)

Procedimiento:

```

gcc -c hello.c -o hello.o
file hello.o

```

Resultado:

```
hello.o: Mach-O 64-bit object arm64
```

Conclusión. `hello.o` es un binario *relocatable* en formato Mach-O (Linux usaría ELF). Ya contiene el código máquina de `hello()`, pero *su dirección sigue siendo 0x0*: los offsets quedan relativos al inicio de cada sección, y será el linker el que les asigne direcciones absolutas al combinarlo con otros objetos.

3. Ejercicio 2 — Compilar `main.c` a `main.o`

Enunciado. Compilar el programa principal hasta el archivo objeto (sin linkear) y analizar qué quedó adentro y qué no.

```

1  /* main.c */
2  #include <stdio.h> /* prototipo de printf */
3
4  extern char* hello(void); /* hello vive en otro modulo */
5
6  int main(void)
7  {
8      printf("%s\n", hello());
9      return 0;
10 }

```

Análisis. Tres elementos importantes:

- `#include <stdio.h>` introduce el *prototipo* de `printf` para que el compilador valide los tipos de los argumentos. La *definición* (el código de `printf`) vive en la libc del sistema.

- `extern char* hello(void)` es solo una *declaración*: le dice al compilador “existe esta función en algún otro `.o`, ya la encontrará el linker”.
- Como consecuencia, `printf` y `hello` van a aparecer en `main.o` como símbolos externos no resueltos (**UND**).

Procedimiento:

```
gcc -c main.c -o main.o
file main.o
```

Resultado:

```
main.o: Mach-O 64-bit object arm64
```

Conclusión. La opción `-c` detiene el proceso tras el ensamblado: preprocesa, compila y ensambla, pero *no* linkea. `main.o` contiene el código de `main` y el literal `"%s\n"`, pero las llamadas a `hello` y `printf` están todavía sin resolver.

Por qué compilar a objetos

Compilar a `.o` en lugar de directamente al ejecutable habilita la **compilación incremental**: si más adelante se modifica `hello.c`, basta con regenerar `hello.o` y volver a linkear, sin tocar `main.c`. Esta es la propiedad que aprovecha `make` y la base de cualquier sistema de *build* moderno.

4. Ejercicio 3 — Análisis de `main.o` y `hello.o`

Enunciado. Inspeccionar a fondo los dos archivos objeto: secciones, tabla de símbolos, tabla de reubicación y desensamblado. La idea es *ver* el resultado de las etapas anteriores y entender por qué hace falta un linker.

3.a Tipo de archivo

Procedimiento:

```
file main.o hello.o
```

Resultado:

```
main.o: Mach-O 64-bit object arm64
hello.o: Mach-O 64-bit object arm64
```

Conclusión. Ambos son objetos relocatables Mach-O para arm64. En GNU/Linux la salida sería “ELF 64-bit LSB relocatable, ARM aarch64”. Las direcciones todavía no están resueltas.

3.b Secciones (`objdump -h`) y tamaños

Procedimiento:

```
objdump -h main.o hello.o
```

Resultado:

```

main.o: file format mach-o arm64
Sections:
Idx Name Size VMA Type
 0 __text 00000040 0000000000000000 TEXT
 1 __cstring 00000004 0000000000000004 DATA
 2 __compact_unwind 00000020 0000000000000048 DATA

hello.o: file format mach-o arm64
Sections:
Idx Name Size VMA Type
 0 __text 0000000c 0000000000000000 TEXT
 1 __cstring 0000000c 000000000000000c DATA
 2 __compact_unwind 00000020 0000000000000018 DATA

```

Análisis.

Objeto	.text	.data	__cstring
main.o	0x40 (64 B)	— (sección ausente)	0x04: "%s\n"
hello.o	0x0c (12 B)	— (sección ausente)	0x0c: "Hello world"

main.o es bastante más grande que hello.o en su sección de código porque tiene prólogo y epílogo de main, armado de argumentos de printf y dos llamadas a funciones (hello, printf). hello(), en cambio, solo arma un puntero y retorna: tres instrucciones, 12 bytes.

Conclusión. No aparece sección __DATA, __data en ninguno de los dos objetos: el programa **no tiene variables globales con valor inicial distinto de cero**. Tampoco hay .bss porque no hay globales inicializadas en cero. El literal "Hello world" no va en .data sino en __cstring porque es de solo lectura.

3.c Contenido de cada sección

Procedimiento:

```
objdump -s main.o hello.o
```

Resultado (parte relevante):

```

main.o:
Contents of section __TEXT,__text:
 0000 ff8300d1 fd7b01a9 fd430091 08008052
 0010 e80b00b9 bfc31fb8 00000094 e8030091
 0020 000100f9 00000090 00000091 00000094
 0030 e00b40b9 fd7b41a9 ff830091 c0035fd6
Contents of section __TEXT,__cstring:
 0040 25730a00 %s..

hello.o:
Contents of section __TEXT,__text:
 0000 00000090 00000091 c0035fd6 ....._
Contents of section __TEXT,__cstring:
 000c 48656c6c 6f20776f 726c6400 Hello world.

```

Lectura. Los bytes 25 73 0a 00 son la secuencia ASCII de "%s\n0": 0x25 = '%', 0x73 = 's', 0x0a = LF, 0x00 = terminador. Los bytes en hello.o forman directamente "Hello world0".

3.d Direcciones de main() y hello()

Procedimiento:

```
objdump --syms main.o hello.o
```

Resultado (parte global):

```
main.o:
 0000000000000000 g F __TEXT,__text _main

hello.o:
 0000000000000000 g F __TEXT,__text _hello
```

Conclusión. Ambas funciones empiezan en el offset 0x0 de su sección `__text`. Esto es lo *normal* en archivos objeto relocatables: cada módulo se piensa como si viviera solo, en su propio espacio que arranca en cero. El linker, más adelante, apila las secciones y reasigna direcciones absolutas únicas.

3.e Dónde se almacena el literal apuntado por `hi`

Recordatorio. `hi` es la macro `#define hi "Hello world"`: *no* es una variable, así que no aparece como símbolo en la tabla. Lo que sí aparece es el literal al que la macro se expande:

```
000000000000000c l 0 __TEXT,__cstring l_.str
```

Conclusión. El string vive en la sección `__cstring` de `hello.o` (de solo lectura, equivalente a `.rodata` en ELF), con etiqueta interna `l_.str`, en el offset 0x0c. La función `hello()` arma esa dirección con `adrp+add` y la devuelve en `x0`.

3.f Tabla de símbolos y tabla de reubicación

Símbolos de `main.o`:

```
0000000000000000 l F __TEXT,__text ltmp0
0000000000000040 l 0 __TEXT,__cstring l_.str
0000000000000040 l 0 __TEXT,__cstring ltmp1
0000000000000048 l 0 __LD,__compact_unwind ltmp2
0000000000000000 g F __TEXT,__text _main
0000000000000000 *UND* _hello
0000000000000000 *UND* _printf
```

Lectura:

- Las columnas son: dirección/offset, tipo (1 local, g global), clase (F function, 0 object/data), sección y nombre.
- `_main` está definido (g global, en `__text`).
- `_hello` y `_printf` aparecen como `*UND*`: el módulo los *usa* pero no los *define*. El linker debe encontrarlos en otro módulo.

Reubicaciones de `main.o`:

```
RELOCATION RECORDS FOR [__text]:
OFFSET TYPE VALUE
000000000000002c ARM64_RELOC_BRANCH26 _printf
0000000000000028 ARM64_RELOC_PAGEOFF12 l_.str
0000000000000024 ARM64_RELOC_PAGE21 l_.str
0000000000000018 ARM64_RELOC_BRANCH26 _hello
```


Cómo leerlo. Cada fila es una entrada *reloc(instr_address, instr_type) → symbol*:

- En el offset 0x18 hay un bl (*branch and link*) cuyo destino debe completarse con la dirección de `_hello`.
- En los offsets 0x24/0x28 hay un par adrp/add que arma la dirección del literal `l_.str`.
- En 0x2c otro bl, esta vez hacia `_printf`.

Reubicaciones de `hello.o`:

```
RELOCATION RECORDS FOR [__text]:
OFFSET TYPE VALUE
0000000000000004 ARM64_RELOC_PAGEOFF12 l_.str
0000000000000000 ARM64_RELOC_PAGE21 l_.str
```

Conclusión. `hello.o` no tiene símbolos **UND** (no llama a nadie); solo tiene dos entradas de reubicación para armar la dirección de su propio literal local `l_.str`.

3.g Las instrucciones call en main()

Procedimiento:

```
objdump -d main.o
```

Resultado (`_main`):

```
0000000000000000 <tmp0>:
 0: sub sp, sp, #0x20
 4: stp x29, x30, [sp, #0x10]
 8: add x29, sp, #0x10
 c: mov w8, #0x0
10: str w8, [sp, #0x8]
14: stur wzr, [x29, #-0x4]
18: bl 0x18 <tmp0+0x18> ; <-- call a _hello (sin resolver)
1c: mov x8, sp
20: str x0, [x8]
24: adrp x0, 0x0 <tmp0>
28: add x0, x0, #0x0
2c: bl 0x2c <tmp0+0x2c> ; <-- call a _printf (sin resolver)
30: ldr w0, [sp, #0x8]
...
3c: ret
```

Lo importante. Los dos bl apuntan “a sí mismos”: sus operandos son 0. Esto coincide exactamente con lo que describen las notas del curso: en el archivo objeto los `call` a símbolos externos quedan sin resolver y la tabla de reubicación guarda la información necesaria para completarlos.

Conclusión. Los operandos no están resueltos en el `.o`: los completará el linker. Esto es exactamente lo que veremos pasar en el ejercicio 5.

3.h Cómo retornan los valores las funciones

Por la convención **AAPCS64** de ARM64:

- El valor de retorno entero o puntero se entrega en `x0` (o `w0` para 32 bits).
- La instrucción `ret` salta a la dirección guardada en `x30` (*Link Register*), donde bl dejó la dirección de retorno.

En el código, `hello()` arma la dirección de la cadena en `x0` y hace `ret`; `main()` guarda el puntero recibido como argumento para `printf` y devuelve el `int 0` en `w0` antes del `ret`. En x86-64 la idea es la misma, pero el registro de retorno sería `rax`.

5. Ejercicio 4 — Generar el ejecutable `myprog`

Enunciado. Combinar `main.o` y `hello.o` en un único ejecutable y entender qué hizo el comando.

Procedimiento:

```
gcc main.o hello.o -o myprog
./myprog
# Hello world
```

¿Se recompilaron los programas?

Respuesta: no. Cuando los argumentos de `gcc` son archivos objeto, `gcc` actúa solo como *driver* del linker. Verificación con `-v`:

```
gcc -v main.o hello.o -o myprog_from_objects
```

Resultado (parte relevante):

```
Apple clang version 17.0.0 ...
".../usr/bin/ld" ... -o myprog_from_objects ... main.o hello.o -lSystem ...
```

No aparecen invocaciones a `cc1` o `as`: el código C ya no se toca. Si los argumentos hubieran sido `main.c` y `hello.c`, la traza mostraría las cuatro etapas (preprocesador, compilador, ensamblador y linker) corriendo para cada `.c`.

Qué hace el linker en concreto

Como se discutió en la introducción, el linking tiene dos partes:

1. **Concatenar secciones y recalcular direcciones.** La sección `__text` final del ejecutable es la concatenación de las `__text` de `main.o` y `hello.o`, con direcciones absolutas únicas.
2. **Resolver referencias externas** usando las tablas de símbolos y reubicación.

En el caso concreto de `myprog`:

- `_main` (de `main.o`) queda en `0x100000460`.
- `_hello` (de `hello.o`) queda en `0x1000004a0`, inmediatamente después.
- El `b1 _hello` de `main` se completa con la dirección real: **linking estático**.
- El `b1 _printf` se enlaza a un *symbol stub* dentro del propio ejecutable que se resolverá dinámicamente en ejecución (ver ejercicio 5).

6. Ejercicio 5 — Análisis assembly de `myprog`

Enunciado. Inspeccionar el ejecutable resultante y verificar qué hizo el linker con cada referencia.

5.a Direcciones de `main()` y `hello()`

Procedimiento:

```
nm -n myprog
```

Resultado:

```
                U _printf
00000000100000000 T __mh_execute_header
00000000100000460 T _main
000000001000004a0 T _hello
```

Lectura. La columna del medio es la clase del símbolo: T = texto/código definido, U = *undefined*.

Conclusión. Ambos símbolos están definidos en el segmento `__TEXT`, uno detrás del otro. El linker hizo lo prometido: tomó las dos secciones `__text` (que arrancaban en offset 0) y las concatenó asignándoles direcciones absolutas únicas. `_printf` sigue siendo U porque su dirección la resuelve dyld en tiempo de ejecución.

5.b ¿La llamada `main -> hello` fue resuelta?

Respuesta: sí, estáticamente.

Procedimiento:

```
objdump -d myprog
```

Resultado (extracto):

```
_main:
100000460: sub sp, sp, #0x20
...
100000478: bl 0x1000004a0 <_hello> ; <-- llamada resuelta
10000047c: mov x8, sp
100000480: str x0, [x8]
100000484: adrp x0, 0x100000000
100000488: add x0, x0, #0x4b8 ; "%s\n"
10000048c: bl 0x1000004ac ; symbol stub for _printf
...
10000049c: ret

_hello:
1000004a0: adrp x0, 0x100000000
1000004a4: add x0, x0, #0x4bc ; "Hello world"
1000004a8: ret
```

Conclusión. El `bl` en `0x100000478` ahora codifica el desplazamiento real al inicio de `_hello`. La invocación quedó *resuelta* porque ambos símbolos están definidos en el mismo ejecutable.

5.c ¿Cómo se resolvió la invocación a `printf()`?

Análisis. `printf` vive en `libSystem.B.dylib` (la `libc` de macOS, equivalente a `libc.so.6` en Linux), una biblioteca *dinámica*. El linker estático no puede poner una dirección absoluta porque el cargador puede ubicar la `libc` en direcciones distintas en cada ejecución (ASLR: *Address Space Layout Randomization*).

La técnica usada es la del **symbol stub** con *lazy binding* descrita en las notas:

1. `ld` genera un stub `_printf$stub` en la sección `__TEXT,__stubs`.
2. `main` hace `bl 0x1000004ac` a ese stub (esa dirección *sí* está en el mismo ejecutable, así que se puede resolver estáticamente).
3. El stub salta indirectamente a través de la *Global Offset Table* (GOT). En Mach-O la sección se llama `__DATA_CONST,__got`; en ELF se llama `.got.plt`.
4. En la primera invocación, la GOT apunta al *dynamic linker* (`dyld`), que resuelve la dirección real de `printf` en `libSystem.B.dylib` y la *reescribe* en la GOT.
5. Las invocaciones siguientes ya encuentran la dirección verdadera y saltan directamente, sin volver a llamar a `dyld`.

Verificación.

```
otool -Iv myprog
otool -L myprog
```

```
Indirect symbols for (__TEXT,__stubs) 1 entries
address index name
0x00000001000004ac 3 _printf
Indirect symbols for (__DATA_CONST,__got) 1 entries
address index name
0x0000000100004000 3 _printf

myprog:
/usr/lib/libSystem.B.dylib (compatibility version 1.0.0, current version 1356.0.0)
```

Conclusión: las dos resoluciones, lado a lado.

Llamada	Resuelve	Cuándo
<code>main -> hello</code>	Linker estático (<code>ld</code>)	En tiempo de <i>linking</i>
<code>main -> printf</code>	Linker dinámico (<code>dyld</code>)	En tiempo de <i>carga</i> / 1. ^a llamada

En Linux es lo mismo, con otros nombres

El mismo mecanismo en ELF/Linux se llama *PLT* (Procedure Linkage Table) en lugar de *symbol stub*, y la GOT también se llama *GOT*. El *dynamic linker* es `/lib64/ld-linux-x86-64.so.2` o equivalente. Las herramientas correspondientes son `ldd` (en vez de `otool -L`) y `readelf -d` (en vez de `otool -l`).

7. Ejercicio 6 — Variables locales: el stack

Enunciado. Demostrar experimentalmente que las variables locales de una función residen en el *stack* del proceso, y no en `.data/.bss/heap`.

Análisis previo. El espacio de direcciones de un proceso ya está organizado como muestra el diagrama de la introducción. Cada invocación a una función crea un *stack frame* con sus parámetros, variables locales, dirección de retorno y registros guardados. El compilador asigna las variables locales como desplazamientos relativos al puntero de marco (`x29/fp` en `arm64`), y el stack **crece hacia direcciones bajas**: cada `call` empuja un nuevo frame hacia abajo.

Idea del experimento.

1. Imprimir las direcciones de varias variables locales declaradas en distintas funciones.
2. Obtener *base* (dirección baja) y *tope* (dirección alta) del stack del hilo principal con las funciones `pthread_get_stackaddr_np` y `pthread_get_stacksize_np`.
3. Verificar que cada dirección de variable local cae dentro del rango [*base*, *tope*).

Para no romper el `main.c` usado en los demás ejercicios, el experimento se hace en una variante: `main_stack.c`.

Código (extracto):

```

1  /* main_stack.c (fragmento) */
2  static void print_inner_locals(uintptr_t stack_base, uintptr_t stack_top)
3  {
4      int inner_a = 10;
5      int inner_b = 20;
6      char inner_buf[16] = "inner";
7      report_local("&inner_a", &inner_a, stack_base, stack_top);
8      report_local("&inner_b", &inner_b, stack_base, stack_top);
9      report_local("inner_buf", inner_buf, stack_base, stack_top);
10 }
11
12 static void print_stack_layout(void)
13 {
14     int local_a = 1;
15     int local_b = 2;
16     double local_c = 3.14;
17     char local_d = 'x';
18     char buffer[32] = "stack-local";
19     pthread_t self = pthread_self();
20     void *stack_top = pthread_get_stackaddr_np(self);
21     size_t stack_size = pthread_get_stacksize_np(self);
22     void *stack_base = (void *)((uintptr_t)stack_top - stack_size);
23     /* imprimir base, tope, tamaño y direcciones de cada local */
24     print_inner_locals((uintptr_t)stack_base, (uintptr_t)stack_top);
25 }

```

Procedimiento:

```
gcc -Wall -Wextra -pedantic main_stack.c hello.c -o myprog_stack
./myprog_stack
```

Resultado. Las direcciones absolutas cambian entre ejecuciones por ASLR, pero la *estructura relativa* se mantiene:

```

Hello world
Stack del hilo principal:
  base (dir baja): 0x16ef98000
  tope (dir alta): 0x16f794000
  tamaño: 8372224 bytes (7.98 MiB)

Locales en main/print_stack_layout:
  &local_a 0x16f7924fc [STACK]
  &local_b 0x16f7924f8 [STACK]
  &local_c 0x16f7924f0 [STACK]
  &local_d 0x16f7924ef [STACK]
  buffer 0x16f792500 [STACK]

Locales en funcion secundaria:

```

```
&inner_a 0x16f79243c [STACK]
&inner_b 0x16f792438 [STACK]
inner_buf 0x16f792450 [STACK]
```

Análisis de la salida.

- Todas las direcciones impresas caen dentro del rango `[stack_base, stack_top)`: las variables locales viven, efectivamente, en el **stack**.
- El stack del hilo principal mide ≈ 8 MiB (8372224 bytes), valor por defecto en macOS y similar al de Linux (`ulimit -s` típicamente da 8192 KiB).
- Las locales de `print_inner_locals` están en direcciones *más bajas* (0x16f79243c) que las de `print_stack_layout` (0x16f7924fc): cada invocación empujó un nuevo frame “hacia abajo”.
- El alineamiento se respeta: `local_c` (un `double`) cae en una dirección múltiplo de 8.

Una variable local con `static` no va al stack

Si declaráramos `static int local_a = 1;` dentro de una función, `local_a` no estaría en el stack: viviría en `.data` (porque tiene valor inicial) como una global de visibilidad restringida. Lo mismo para variables globales fuera de cualquier función. Es importante distinguir *duración de almacenamiento* (automática, estática, dinámica) de *scope*.

Respuesta concreta. Las variables locales se asignan en el *stack* del proceso, una región del espacio de direcciones que crece hacia direcciones bajas y se organiza en *stack frames* (uno por invocación de función). ASLR aleatoriza las direcciones absolutas en cada ejecución, pero la estructura relativa (frames internos por debajo de los externos) permanece.

8. Ejercicio 7 — Biblioteca estática `libhello.a`

Enunciado. Empaquetar `hello.o` y un segundo módulo `hello2.o` en una biblioteca estática, y enlazar un programa contra ella.

Análisis. Una biblioteca **estática** es un *archive* (formato `.a`) que agrupa varios `.o` junto con un *índice de símbolos*. El linker, ante una referencia sin resolver, busca en el índice del `.a` y, si lo encuentra, **copia el `.o` correspondiente al ejecutable**.

Para no romper `main.c`, este ejercicio usa una variante (`main_lib.c`) y un segundo módulo (`hello2.c`):

```
1 /* hello2.c */
2 #define hi2 "Hello from hello2"
3 char *hello2(void) { return hi2; }
```

```
1 /* main_lib.c */
2 #include <stdio.h>
3 extern char *hello(void);
4 extern char *hello2(void);
5 int main(void) {
6     printf("%s\n", hello());
7     printf("%s\n", hello2());
8     return 0;
9 }
```

7.a Crear la biblioteca estática

Procedimiento:

```
gcc -Wall -Wextra -pedantic -c hello.c -o hello.o
gcc -Wall -Wextra -pedantic -c hello2.c -o hello2.o
ar rcs libhello.a hello.o hello2.o
```

Análisis de las opciones de ar.

- **r** *reemplaza* miembros existentes (o los agrega si no existían).
- **c** *crea* el archivo si no existe (silencia el warning).
- **s** genera o actualiza el *índice de símbolos*. Es lo que permite al linker responder rápidamente “¿qué objeto define a **hello**?”.

7.b Listar los módulos de la biblioteca

Procedimiento:

```
ar -t libhello.a
```

Resultado:

```
__SYMDEF SORTED
hello.o
hello2.o
```

Conclusión. La primera entrada (`__SYMDEF SORTED`) es el índice de símbolos interno generado por `ar -s`; los módulos reales son `hello.o` y `hello2.o`. En ELF/Linux el índice equivalente se llama `/SYMDEF` o `__index` según implementación.

7.c Compilar y linkear contra la biblioteca

Procedimiento:

```
gcc -Wall -Wextra -pedantic main_lib.c -L. -lhello -o myprog_lib
./myprog_lib
```

Resultado:

```
Hello world
Hello from hello2
```

Detalle del comando.

- `-L.` agrega el directorio actual al *path* de búsqueda de bibliotecas.
- `-lhello` pide buscar una biblioteca llamada `libhello`. El prefijo `lib` y el sufijo (`.a` / `.dylib` / `.so`) los pone el propio linker.

Si conviven `.a` y `.dylib`, gana la dinámica

Si en el mismo directorio existieran `libhello.a` y `libhello.dylib`, `ld` prefiere por defecto la dinámica. Es lo que ocurre en este directorio: al compilar con `-lhello` el linker elige

`libhello.dylib`, que solo exporta `hello`, y la compilación falla por `hello2` no encontrado. Para forzar la estática hay dos opciones: pasar `libhello.a` directamente en la línea de comandos (`gcc main_lib.c libhello.a`), o usar la flag `-Wl,-search_paths_first` con `libhello.a` explícita. En Linux la flag equivalente es `-Wl,-Bstatic ... -Wl,-Bdynamic`.

Conclusión. El linker, al ver que `main_lib.o` referencia `_hello` y `_hello2`, consulta el índice de `libhello.a`, encuentra que provienen de `hello.o` y `hello2.o`, **extrae esos miembros y los integra** en el ejecutable. El resultado es un binario *autocontenido*: una vez linkeado no necesita la biblioteca para correr.

9. Ejercicio 8 — Biblioteca dinámica y `dlopen`

Enunciado. Generar una biblioteca dinámica con la función `hello`, escribir un programa que la cargue *en tiempo de ejecución* con `dlopen/dlsym`, y verificar que el ejecutable *no* contiene el código de `hello`.

Análisis. Las bibliotecas **dinámicas** (`.so/.dylib/.dll`) son archivos que el SO carga y enlaza bajo demanda. El ejecutable no incluye su código: si varios procesos las usan, una sola copia en memoria los sirve a todos.

Existen dos formas de usarlas:

- **Linking dinámico implícito** (lo que vimos con `printf` en el ejercicio 5): el linker estático genera *stubs* y entradas en la GOT al momento de enlazar el ejecutable. El *dynamic linker* resuelve los símbolos al cargar el programa.
- **Carga dinámica explícita** (este ejercicio): el programa decide en tiempo de ejecución qué biblioteca cargar, usando la API POSIX `<dlfcn.h>` (`dlopen/dlsym/dlclose`). Es la base de los *plugins* y de cualquier mecanismo de extensibilidad en ejecución.

8.a Generar la biblioteca dinámica

Procedimiento:

```
gcc -Wall -Wextra -pedantic -fPIC -shared hello.c -o libhello.dylib
file libhello.dylib
```

Resultado:

```
libhello.dylib: Mach-O 64-bit dynamically linked shared library arm64
```

Por qué cada flag importa.

- `-fPIC` (*Position Independent Code*) es obligatorio: la biblioteca puede cargarse en distintas direcciones en cada proceso (ASLR), así que el código no puede asumir una dirección de carga fija. Toda referencia a un dato global pasa por la GOT.
- `-shared` le indica a `gcc` que la salida es una biblioteca compartida, no un ejecutable.

Verificación de los símbolos exportados:

```
nm libhello.dylib
```

```
0000000000000320 T _hello
```


La biblioteca exporta exactamente un símbolo: `_hello`.

8.b/8.c Carga dinámica con `dlopen`

Código:

```
1  /* main_dlopen.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <dlfcn.h>
5
6  typedef char *(*hello_fn_t)(void);
7
8  int main(void)
9  {
10     const char *lib_path = "./libhello.dylib";
11     void *handle = dlopen(lib_path, RTLD_NOW);
12     if (handle == NULL) {
13         fprintf(stderr, "dlopen: %s\n", dlerror());
14         return 1;
15     }
16     dlerror(); /* limpia el estado de error previo */
17
18     hello_fn_t hello_fn = NULL;
19     *(void **>(&hello_fn) = dlsym(handle, "hello");
20     const char *err = dlerror();
21     if (err != NULL) {
22         fprintf(stderr, "dlsym: %s\n", err);
23         dlclose(handle);
24         return 1;
25     }
26
27     printf("%s\n", hello_fn());
28
29     dlclose(handle);
30     return 0;
31 }
```

Decisiones de diseño.

- `RTLD_NOW` fuerza la resolución de todos los símbolos al cargar la biblioteca. `RTLD_LAZY` los resolvería bajo demanda; para una biblioteca pequeña es indistinto.
- `*(void **>(&hello_fn) = dlsym(...)` es el modismo recomendado por POSIX para evitar el *warning* sobre conversión de `void *` a puntero a función (en C estándar esa conversión no está garantizada, aunque en POSIX sí lo está).
- Se llama a `dlerror()` antes y después de `dlsym` para no confundir un símbolo con valor legítimamente `NULL` con un símbolo *no encontrado*.

Compilación y ejecución (en macOS la `libc` provee `dlopen` de fábrica; en Linux haría falta agregar `-ldl`):

```
gcc -Wall -Wextra -pedantic main_dlopen.c -o myprog_dlopen
./myprog_dlopen
```

```
Hello world
```

8.c El código de `hello` no está en el ejecutable

Procedimiento:

```
otool -L myprog_dlopen
nm myprog_dlopen | grep hello
```

Resultado:

```
myprog_dlopen:
  /usr/lib/libSystem.B.dylib (compatibility version 1.0.0, current version 1356.0.0)

(la búsqueda de "hello" en nm no devuelve nada)
```

Conclusión. La única dependencia es `libSystem.B.dylib` (que provee `printf`, `dlopen`, etc.). No hay referencia a `libhello.dylib` porque el ejecutable nunca fue enlazado contra ella; la cargará el propio programa con `dlopen`. Y `nm` no encuentra el símbolo `_hello` porque ese código no está en el binario: vive en la biblioteca dinámica y se carga en ejecución.

8.d Sección de enlazado dinámico

En Linux se usaría `readelf -d myprog_dlopen` para ver la sección `.dynamic`. En macOS el equivalente es `otool`:

```
otool -L libhello.dylib
```

```
libhello.dylib:
  libhello.dylib (compatibility version 0.0.0, current version 0.0.0)
  /usr/lib/libSystem.B.dylib ...
```

La biblioteca declara su propio nombre como *install name* y a su vez depende de la `libc` (`libSystem.B.dylib`).

Conexión con la teoría

Implícito vs explícito

- **Linking dinámico implícito** (ejercicios 4 y 5): el linker estático genera stubs y entradas en la GOT, y el *dynamic linker* resuelve los símbolos en tiempo de carga sin que el programa tenga que hacer nada.
- **Carga dinámica explícita** (este ejercicio): el propio programa decide qué biblioteca cargar y cuándo, mediante `dlopen/dlsym`. Es lo que usan los sistemas de *plugins*, los lenguajes interpretados cuando cargan extensiones, y las extensiones del kernel (módulos).

10. Ejercicio 9 — Automatización con Makefile

Enunciado. Empaquetar la construcción de `myprog` en un `Makefile` que permita compilación incremental, y verificar que `make` solo reconstruye lo que cambió.

Análisis. En un proyecto real, los pasos del toolchain no se invocan a mano: se automatizan con un *build system*. El más clásico en UNIX es **GNU make**, basado en un `Makefile` que describe

objetivos (targets), sus *dependencias* y los *comandos* para construirlos. La forma básica de una regla es

```
target: dependencies
<TAB>command
```

`make` construye un grafo de dependencias y, comparando los *timestamps* de cada archivo, decide qué reglas disparar. Si el objetivo es más nuevo que todas sus dependencias, no se reconstruye: de ahí la **compilación incremental**.

Las **variables automáticas** clave son:

- `$@`: nombre del target.
- `$^`: todas las dependencias.
- `$<`: la primera dependencia.

Y las **reglas de patrón** (`%.o: %.c`) evitan repetir la misma acción para cada archivo fuente.

9.1 Makefile usado

Los originales `main.c` y `hello.c` viven en el directorio del práctico. Para mantener la convención de los demás ejercicios, el `Makefile` se ubica dentro de `ejercicio-9/` y referencia los fuentes vía la variable especial `VPATH`.

```
# VPATH le dice a make donde buscar los archivos fuente cuando no
# estan en el directorio actual.
VPATH = ..

CC = gcc
CFLAGS = -Wall -Wextra -pedantic

# Objetivo por defecto: myprog
myprog: main.o hello.o
    $(CC) $(CFLAGS) -o $@ $^

main.o: main.c
    $(CC) $(CFLAGS) -c $< -o $@

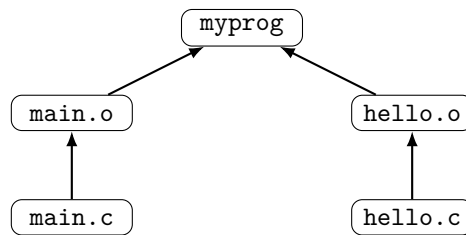
hello.o: hello.c
    $(CC) $(CFLAGS) -c $< -o $@

.PHONY: clean
clean:
    rm -f main.o hello.o myprog
```

Decisiones de diseño.

- `VPATH = ..` es la forma idiomática de separar fuentes y artefactos de compilación: `make` encuentra `main.c/hello.c` un nivel arriba y produce los `.o` y el ejecutable en el directorio actual.
- `CC` y `CFLAGS` centralizan las opciones del compilador.
- `$@`, `$^`, `$<` hacen las reglas concisas y reutilizables.
- `.PHONY: clean` declara `clean` como objetivo “ficticio”: si existiera un archivo llamado `clean`, sin esta declaración `make` lo consideraría “ya construido” y no ejecutaría la regla.

9.2 Grafo de dependencias



`make myprog` requiere `main.o` y `hello.o`; cada `.o` requiere su `.c`. `make` recorre el grafo en post-orden: primero compila los `.o` que falten o estén desactualizados y solo entonces invoca al linker.

9.3 Ejecución desde cero

```
$ make clean
rm -f main.o hello.o myprog

$ make
gcc -Wall -Wextra -pedantic -c ../main.c -o main.o
gcc -Wall -Wextra -pedantic -c ../hello.c -o hello.o
gcc -Wall -Wextra -pedantic -o myprog main.o hello.o

$ ./myprog
Hello world
```

9.4 Compilación incremental

Si se vuelve a invocar `make` sin tocar nada, el ejecutable ya está al día y `make` no ejecuta ningún comando:

```
$ make
make: `myprog' is up to date.
```

Si se modifica solo un archivo fuente, `make` rehace **únicamente lo necesario**:

```
$ touch ../hello.c
$ make
gcc -Wall -Wextra -pedantic -c ../hello.c -o hello.o
gcc -Wall -Wextra -pedantic -o myprog main.o hello.o
```

`main.o` no se recompiló porque `main.c` no cambió. En proyectos grandes, recompilar todo desde cero es inviable; `make` escala porque solo toca lo que tiene que tocar.

9.5 Versión compacta con regla de patrón

La misma funcionalidad puede escribirse aún más breve usando una regla de patrón `%.o : %.c`, que define cómo construir *cualquier* `.o` a partir de un `.c`:

```
VPATH = ..
CC = gcc
CFLAGS = -Wall -Wextra -pedantic

myprog: main.o hello.o
    $(CC) $(CFLAGS) -o $@ $^
```

```
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

.PHONY: clean
clean:
    rm -f main.o hello.o myprog
```

La versión con reglas explícitas (la que está activa) es más didáctica porque hace visibles los tres comandos del toolchain. La versión con patrón es la que se usa en proyectos reales porque escala sin repetir reglas.

Conclusión. El **Makefile** no agrega capacidades nuevas al proceso de compilación; *automatiza* exactamente las mismas etapas vistas en los ejercicios 1–4. Lo que aporta es *automatización con conocimiento de dependencias*, la herramienta que hace manejables los proyectos grandes manteniendo compilaciones rápidas y reproducibles.

Cierre

A lo largo del práctico se “abrió” el proceso que normalmente queda oculto detrás del comando `gcc -o myprog ...`:

- Pre-procesamiento, compilación y ensamblado generan los artefactos intermedios (`.i`, `.s`, `.o`) inspeccionables con `objdump` / `otool` / `nm`.
- Los archivos objeto contienen secciones (`.text`, `.data`, `.bss`, `.rodata/__cstring`), tabla de símbolos y tabla de reubicación; las funciones empiezan en offset `0x0` y los `call` a símbolos externos quedan sin resolver.
- El linker concatena las secciones, recalcula direcciones y resuelve referencias: *estáticamente* las que se conocen (como `hello`) y *dinámicamente* las que viven en bibliotecas compartidas (como `printf`, vía `stubs/PLT` y `GOT`).
- Las variables locales se asignan en el *stack*, una región del proceso que crece hacia direcciones bajas; se verificó experimentalmente que las direcciones impresas caen dentro del rango reportado por `pthread_get_stackaddr_np`.
- Las bibliotecas estáticas (`libhello.a`) son contenedores `ar` de `.o` con índice de símbolos; el linker extrae solo los miembros necesarios y los integra al ejecutable.
- Las bibliotecas dinámicas (`libhello.dylib/libhello.so`) se cargan en tiempo de ejecución, ya sea de forma implícita (vía `stubs` y `GOT`) o explícita (vía `dlopen/dlsym`).
- `make` automatiza todo el flujo: un **Makefile** declara objetivos, dependencias y comandos, y `make` reconstruye solo lo que cambió comparando *timestamps*.

Esta maquinaria es la base de los conceptos de los prácticos siguientes: cómo el SO carga un ejecutable en memoria, cómo organiza el espacio de direcciones del proceso y cómo distintos procesos pueden compartir código mediante bibliotecas dinámicas.

Referencias

1. Marcelo Arroyo. *Notas del curso de Sistemas Operativos*, capítulo *Herramientas de desarrollo*. UNRC, 2026. <https://marceloarroyo.gitlab.io/cursos/SO/index.html#/toolchain>
2. Brian Kernighan, Dennis Ritchie. *The C Programming Language*, 2.^a edición. Prentice Hall.
3. John R. Levine. *Linkers and Loaders*. Morgan Kaufmann, 2000.

4. Páginas de manual: `man 1 gcc`, `man 1 ld`, `man 1 ar`, `man 1 objdump`, `man 1 otool`, `man 1 nm`, `man 1 readelf`, `man 1 ldd`, `man 3 dlopen`.